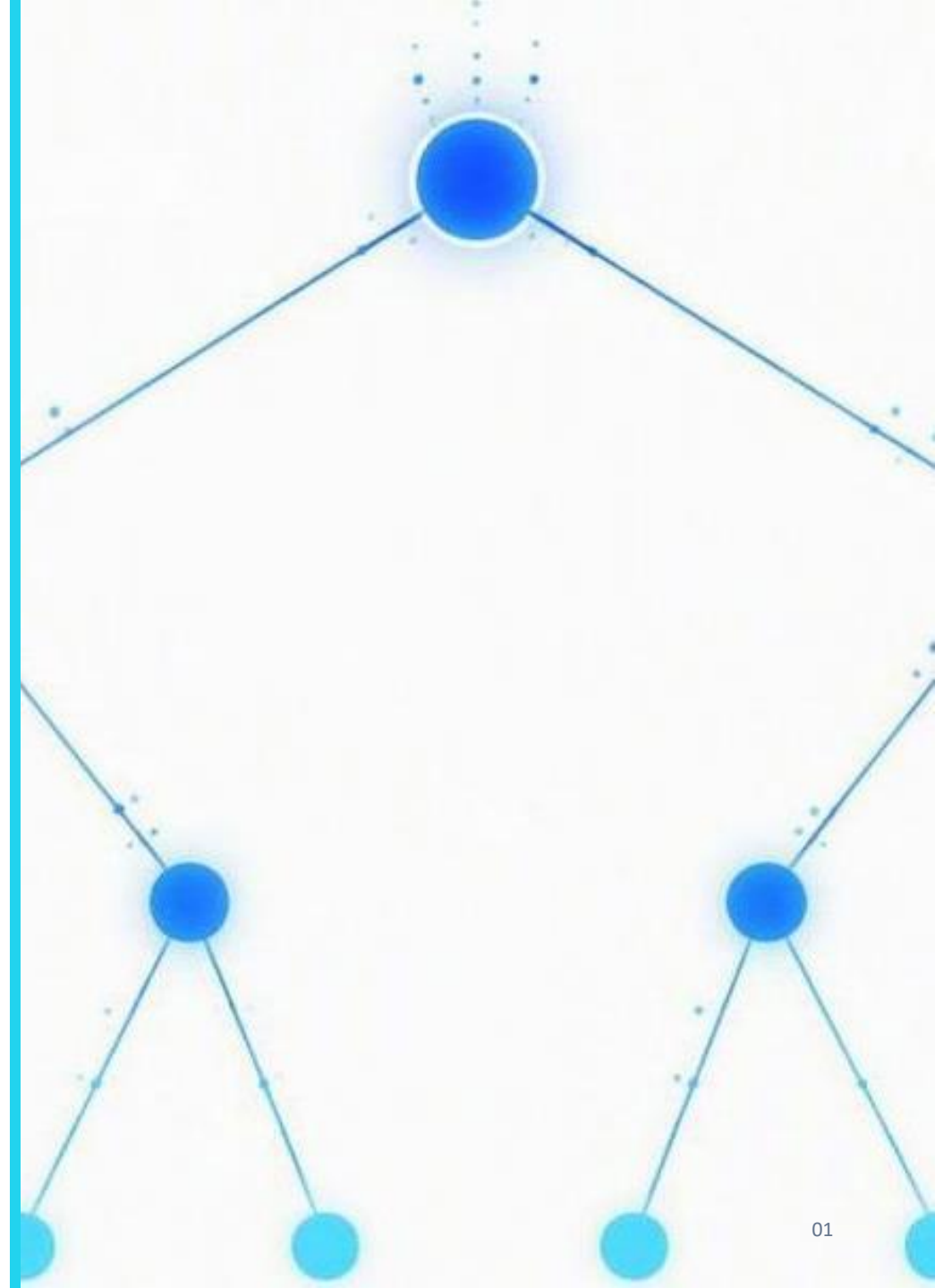


CIENCIA DE DATOS · APRENDIZAJE AUTOMÁTICO

Modelos de Machine Learning

De los árboles de decisión a la validación de modelos predictivos

Guía práctica · Nivel intermedio · Ejemplos en Python



INTRODUCCIÓN

Aprender a predecir precios, igual que un experto

Tu primo hizo una fortuna especulando con propiedades. Te propone asociarse: **él pone el capital y vos ponés los modelos** que predicen cuánto vale cada casa.

¿Cómo lo hacía él? Por “intuición”. Pero en realidad identificó patrones de precios en casas que vio antes, y los aplica a casas nuevas.

El machine learning funciona igual

Capta patrones a partir de datos y los usa para predecir casos nuevos. Empezamos con el **árbol de decisión**: simple de entender y base de algunos de los mejores modelos.



El modelo más simple posible

Divide las casas en solo dos categorías. El precio predicho para cualquier casa es el precio promedio histórico de las casas de su misma categoría.

- **Ajustar (fit / train)** Usamos los datos para decidir cómo dividir las casas y qué precio predecir en cada grupo.
- **Datos de entrenamiento** Son los datos con los que se ajusta el modelo y de los que aprende sus patrones.
- **Predecir** Una vez ajustado, aplicamos el modelo a casas nuevas para estimar su precio.



Capturar patrones de los datos se llama ajustar o entrenar el modelo.

Más divisiones capturan más factores

Un árbol con más **divisiones** (“**splits**”) es más **profundo** y puede considerar otros factores: baños, tamaño del terreno, ubicación...

Hoja (leaf)

El punto final donde el árbol hace una predicción. Seguí el camino según las características de la casa hasta llegar a una hoja.

A mayor profundidad, más grupos: con 10 divisiones se llega a $2^{10} = 1.024$ hojas.



Depth-2 · el precio predicho está en la hoja

Primer paso: familiarizarse con Pandas

Pandas es la herramienta principal para explorar y manipular datos. Su estructura clave es el **DataFrame**: una tabla, como una hoja de cálculo.

- **import pandas as pd**
Se importa abreviado como pd por convención.
- **pd.read_csv(ruta)**
Carga los datos del archivo en un DataFrame.
- **df.describe()**
Resume estadísticamente cada columna numérica.



```
# Guardar la ruta del archivo
melbourne_file_path = '../melb_data.csv'

# Leer los datos en un DataFrame
melbourne_data = pd.read_csv(melbourne_file_path)

# Resumen estadístico de los datos
melbourne_data.describe()
```

	Rooms	Price	Landsize
count	13580	13580	13580
mean	2.94	1.08e6	558.4
min	1.00	85000	0.0
max	10.00	9.0e6	433014.0

Qué significan los 8 números de describe()

Métrica	Rooms	Price	Landsize
count	13.580	13.580	13.580
mean	2,94	1,08 M	558,4
std	0,96	639.310	3.990,7
min	1	85.000	0
25%	2	650.000	177
50%	3	903.000	440
75%	3	1,33 M	651
max	10	9,00 M	433.014

count

Cuántas filas tienen valores no faltantes (no todas las casas registran todo).

mean • std

El promedio y cuánto se dispersan los valores alrededor de él.

min • max

El valor más bajo y el más alto de la columna.

25% • 50% • 75%

Percentiles: al ordenar, el 25% deja por debajo una cuarta parte de los datos.

El objetivo (y) y las características (X)

y · el objetivo (prediction target)

La columna que queremos predecir. Se selecciona con notación de punto.

```
y = melbourne_data.Price
```

X · las características (features)

Las columnas que se ingresan al modelo. Se eligen con una lista entre corchetes.

```
X = melbourne_data[features]
```

```
# Elegimos algunas columnas con nuestra intuición
melbourne_features = ['Rooms', 'Bathroom', 'Landsize', 'Lattitude', 'Longtitude']
X = melbourne_data[melbourne_features] # revisar con X.describe() y X.head()
```

Cuatro pasos con scikit-learn

1 Definir

¿Qué tipo de modelo? Un árbol de decisión y sus parámetros.

2 Ajustar

Capturar los patrones de los datos. El corazón del modelado.

3 Predecir

Estimar el precio de casas nuevas.

4 Evaluar

Medir qué tan precisas son las predicciones.

```
from sklearn.tree import DecisionTreeRegressor

# Definir (random_state fija resultados reproducibles)
melbourne_model = DecisionTreeRegressor(random_state=1)

# Ajustar el modelo
melbourne_model.fit(X, y)
```

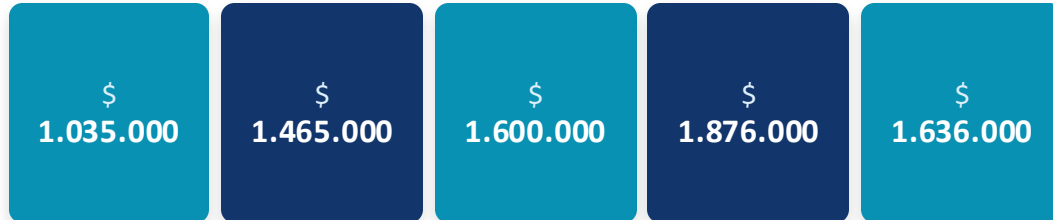
random_state

Muchos modelos usan algo de aleatoriedad al entrenar. Fijar un número garantiza los mismos resultados en cada ejecución: es buena práctica y el valor exacto no cambia la calidad.

Aplicar el modelo ya ajustado

En la práctica querés predecir casas **nuevas**, pero acá predecimos las primeras 5 filas del entrenamiento para ver cómo funciona **.predict()**.

Precios predichos para las 5 primeras casas



Casa 1 → 2 hab · 1 baño · 156 m² de terreno

```
print("Predicciones para 5 casas:")
print(X.head())
print("Las predicciones son:")
print(melbourne_model.predict(X.head()))

>> Las predicciones son:
[1035000.  1465000.  1600000.
 1876000.  1636000.]
```

El error absoluto medio (MAE)

Construiste un modelo... ¿pero qué tan bueno es? Necesitamos resumir la calidad en una sola métrica.

$$\text{error} = \text{actual} - \text{predicho}$$

MAE: tomamos el **valor absoluto** de cada error (lo hace positivo) y promediamos. En palabras simples:

“En promedio, nuestras predicciones se desvían unos X.”

Predicho vs. real

Real \$150.000

Predicho \$100.000

error = \$50.000

Ejemplo de error

Si una casa cuesta \$150.000 y predijiste \$100.000, el error es \$50.000.

MAE dentro de la muestra ≈ \$434

Medir con los mismos datos engaña

Si usamos una sola muestra para construir y evaluar el modelo, sobreestimamos su calidad.

1

Un patrón casual

En tu muestra, todas las casas con puerta verde eran carísimas... por pura casualidad.

2

El modelo lo aprende

Su trabajo es hallar patrones, así que siempre predice precios altos para puertas verdes.

3

Parece muy preciso

Como el patrón viene del entrenamiento, el modelo luce exacto... en esos mismos datos.

4

Falla con datos nuevos

Si el patrón no se repite, el modelo es inútil en la práctica.

Solución: medir el rendimiento sobre datos que el modelo no vio al entrenar — los **datos de validación**.

train_test_split: la prueba real

```
from sklearn.model_selection import train_test_split

# separar entrenamiento y validación
train_X, val_X, train_y, val_y = \
    train_test_split(X, y, random_state=0)

model.fit(train_X, train_y)
preds = model.predict(val_X)
mean_absolute_error(val_y, preds)
```

Dentro de la muestra (in-sample)

\$500 de error

Fuera de la muestra (validación)

\$265.807 de error

De casi exacto a inservible. El error en datos nuevos es $\sim 1/4$ del valor promedio de una vivienda (\$1,1 M).

Underfitting vs. Overfitting

Overfitting · sobreajuste

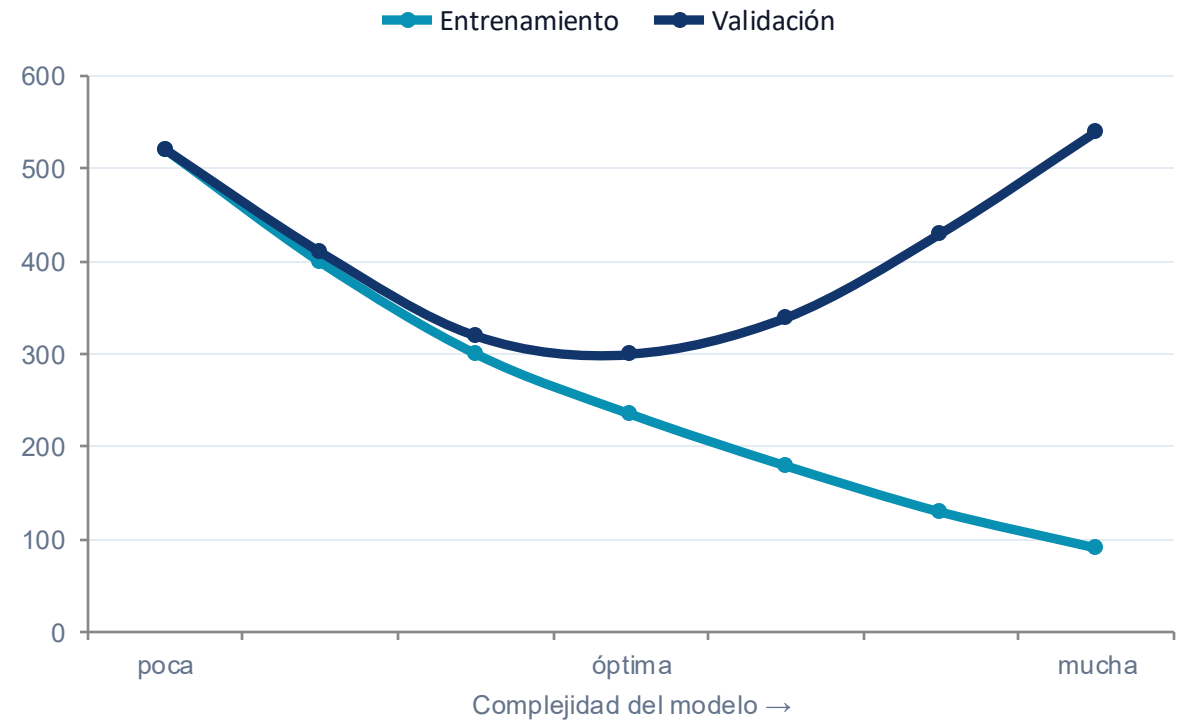
Árbol muy profundo: memoriza el entrenamiento casi perfecto, pero falla con datos nuevos. Cada hoja tiene muy pocas casas.

Underfitting · subajuste

Árbol muy poco profundo: no distingue grupos y falla incluso en el entrenamiento. No capta patrones importantes.

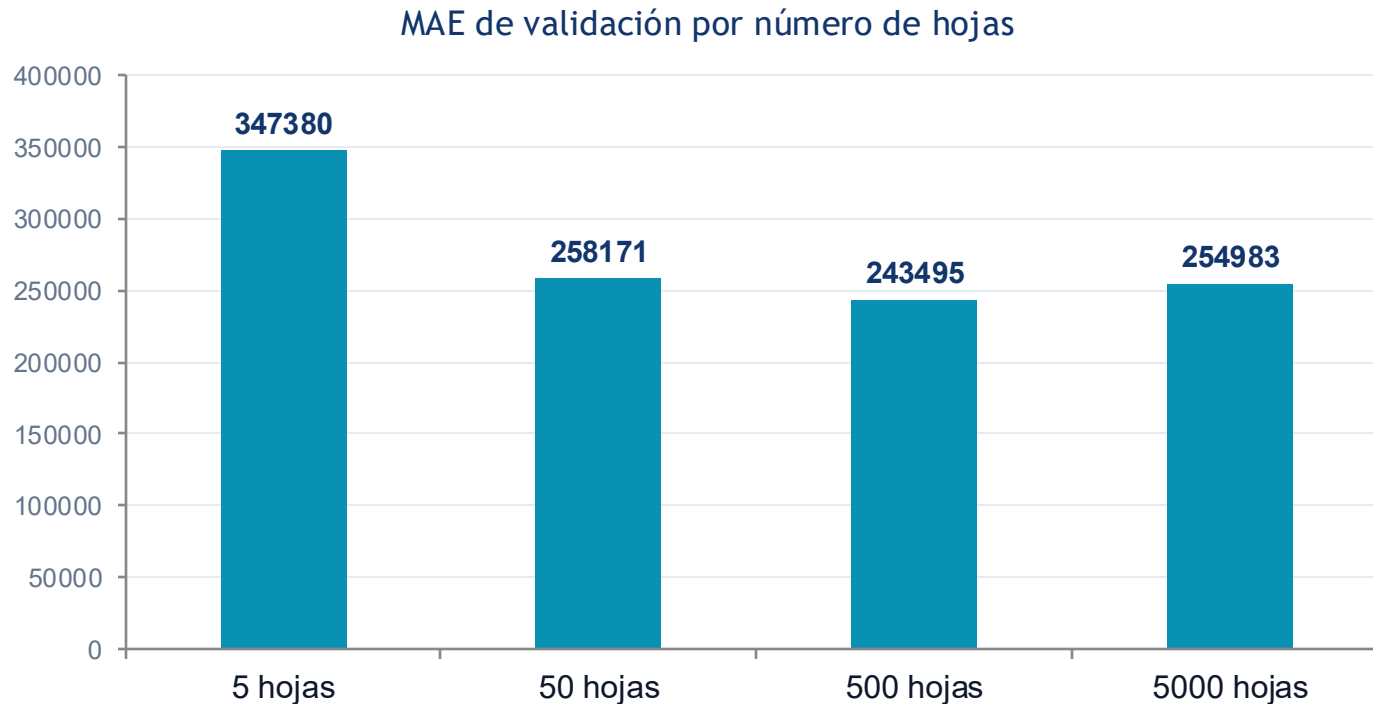
Buscamos el punto justo: el mínimo de la curva de validación.

Error (MAE) según la complejidad del árbol



Ajustar max_leaf_nodes

max_leaf_nodes controla el equilibrio: más hojas nos mueven del subajuste hacia el sobreajuste. Comparamos el MAE de validación:



ÓPTIMO

500 hojas

Menor error de validación: \$243.495

Cómo elegirlo

Un bucle for prueba varios valores de `max_leaf_nodes` con una función `get_mae()` y nos quedamos con el de menor error de validación.

EN RESUMEN

Puntos clave

1 Modela como un experto

El árbol de decisión capta patrones de los datos para predecir casos nuevos.

2 Explorá y elegí bien

Con Pandas conocés los datos; definís el objetivo (y) y las características (X).

3 Validá siempre

El MAE dentro de la muestra engaña: medí sobre datos de validación no vistos.

4 Buscá el punto justo

Ni subajuste ni sobreajuste; afiná la profundidad con `max_leaf_nodes`.

El árbol de decisión es la base de algunos de los mejores modelos de la ciencia de datos.